CSC 2770 Honors Project 1
Simon Glashauser
Spatial and Temporal Locality

Project Resources:

The source code for this project is available on GitHub at https://github.com/sglasha/Exploration-on-Spatial-and-Temporal-Locality. The repository contains the Python program used to measure and visualize the performance differences described in this paper, along with all generated figures. Information on how to run the program is in the readme file.

Introduction:

Computers use cache memory to help speed up the performance because of the quick access time. The CPU uses spatial and temporal locality to access data more efficiently without having to go to main memory. This matters because it takes more time to access main memory than it does to access the CPU's cache, and when using large sets of data it begins to become important to optimize memory usage. This paper will showcase two examples of spatial locality and one example of temporal locality, and benchmark each using a Python program to measure and visualize the performance differences.

1. Spatial and Temporal Locality:

1.1 Spatial Locality

Spatial Locality refers to the tendency of a program to access memory in close proximity to recently accessed data. This is done by having cache load lines of data instead of single values to allow the program to access the memory already in the cache line without taking the extra cycles required to access main memory.

1.1.1 Sequential vs Strided Array Traversal:

Sequential traversal is visiting each element in order, which maximizes spatial locality as per its definition. Strided traversal is visiting each element while skipping k elements, where k is an arbitrary number that causes programs to not leverage the advantages of spatial locality at large values. For example, stride 2 won't have nearly as much of an impact as stride 1024.

Strided Array Traversal Graph: Figure 1

1.1.2 Row-Wise vs Column-Wise 2D Array Traversal:

Row Wise Traversal accesses each element in the corresponding row sequentially before moving onto the next row while Column Wise Traversal accesses each element in the corresponding column before moving onto the next column. Because most programming languages tend to store 2D arrays in row major order, Row Wise Traversal leverages the advantages of spatial locality while Column Wise Traversal doesn't. This applies to languages such as C, C++, Java, and Python, but languages such as Fortran, Matlab, Julia, and R use column major order meaning that the opposite is true.

Row-Wise vs Column-Wise Array Traversal: Figure 2

1.2 Temporal Locality:

Temporal Locality refers to the tendency of a program to access the same location in memory repeatedly within a short period of time. This is done by having cache store recently accessed data. The cache typically

manages this using a Least Recently Used eviction policy, meaning data that has not been accessed recently is removed to make room for new data.

1.2.1 Recursive Access

Recursive Access will access the same element repeatedly over a large number of times, and will showcase the strengths of Temporal locality in accessing the same element repeatedly. Accessing the same data from the new variable will add time.

Temporal Locality shown through Recursive Access: Figure 3

2. Results:

On an array size of 1000^2 and a 1000 by 1000 element matrix, I got the following results in seconds.

Test	Time (ms)
Sequential Traversal	24.9964799877489
Strided Traversal	42.96550499566365
Row Wise Iteration	101.14789000363089
Column Wise Iteration	104.55485001439228
Access Element 10000 Times	1.2089118699892423
Recursively Access Element 10000 Times	2.255573850107612

This shows that it takes more time to do strided traversal than sequential traversal. It also shows how column wise iteration takes more time than row wise iteration, and how temporal locality affects the output with the average times being displayed.

Graphs

Figure 1:

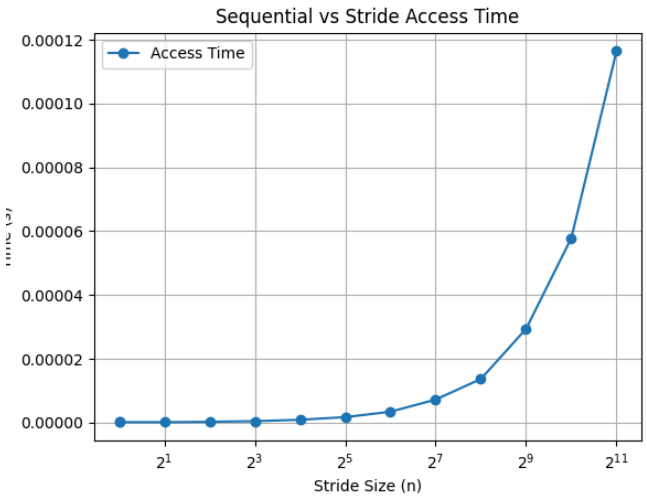


Figure 2:

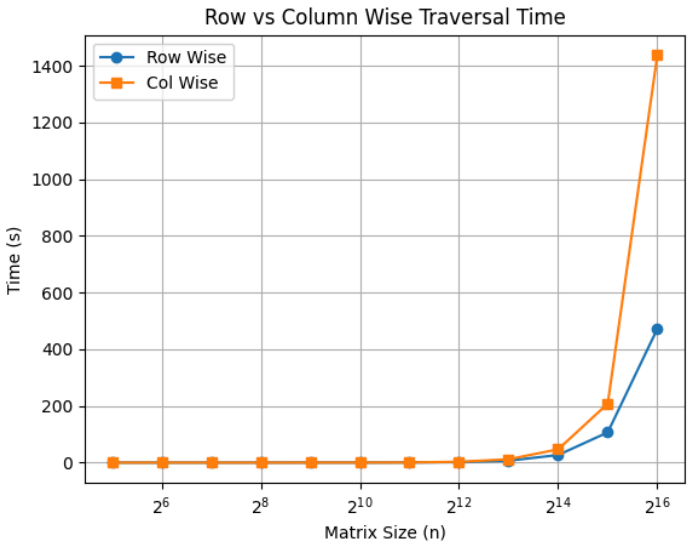
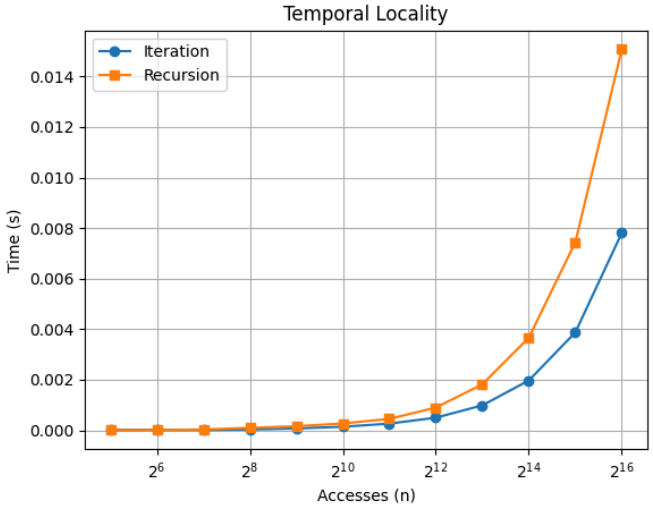


Figure 3:



3. Conclusion:

Spatial Locality heavily affects how quickly the program runs especially on large sets of data as shown in figure 2. Temporal locality also affects the program since figure 3 is the average time it takes to be accessed n times, but it took less time to run than figure 2.